

UNIVERSITÀ DI PISA

Decentralised Lending Service - Project Report

Marco Pernisco
Student ID: 683674

1. Implementation Choices of Smart Contracts

The Decentralised Lending Service is built on Ethereum utilizing a modular and upgradeable smart contract architecture. The main components and design choices are as follows:

- **Upgradeability via UUPS Proxy**: The main `FundingPool.sol` contract implements the Universal Upgradeable Proxy Standard (UUPS). This ensures that while the core state is maintained in a proxy contract (`FundingPoolProxy.sol`), the underlying implementation logic can be upgraded without migrating the accumulated capital or losing historical loan data.
- **Factory Pattern for Loans**: Instead of managing all loan lifecycles internally within the pool, the system uses a factory pattern. When a loan proposal is successfully approved, the FundingPool dynamically deploys a dedicated `LoanContract.sol`. This cleanly isolates the state and balance of each active loan and ensures that operations like repayment or default evaluations are self-contained.
- **Dynamic Voting and Capital Allocation**: A strict voting period is enforced (12 blocks). Voting power is directly proportional to a contributor's current *unlocked* (disposable) balance at the time of resolution. If a loan is approved, the required principal is locked proportionally across all unlocked funds of the participating contributors, and any rounding dust is absorbed into the loan principal rather than being unfairly assigned.
- **Compensation Pool & Collateral Logic**: To incentivize repayment and protect against defaults, a portion of the interest paid on successful loans is directed to a compensation pool. In case of a default, contributors can request compensation from this pool, which dynamically adjusts the global collateral percentage (increasing it on failures, decreasing it on successes) to balance the risk-reward ratio of the protocol.
- **Cross-Chain Oracle Integration**: A centralized, trusted `BitcoinOracle` contract is deployed to track verified Bitcoin balances. The protocol delegates the off-chain balance fetching to a Python service that parses Bitcoin blocks and pushes the balance back on-chain. This provides a non-locked "proof of liquidity" used as a prerequisite to pass a loan proposal.

2. Gas Evaluation Results

Gas profiling was performed locally on the Hardhat testing environment. The measured gas costs for the main protocol operations are:

Operation	Gas Used	Description
`contribute()`	140,923	Depositing ETH into the funding pool
`withdraw()`	47,670	Withdrawing available unlocked funds
`createLoanProposal()`	189,788	Submitting a new loan request
`voteOnProposal()`	106,758	Casting a weighted vote on a proposal
`requestUpdate()`	24,575	Requesting an oracle balance check
`update()`	48,799	Oracle pushing BTC balance on-chain
`executeApprovedLoan()`	912,836	Resolving vote, locking funds, and deploying `LoanContract`
`repay()`	183,475	Processing a loan repayment and distributing interest
`claimInterestGain()`	33,454	Contributor claiming accrued interest

Note: `executeApprovedLoan` is significantly more expensive as it handles complex proportional math across all contributors and deploys a new smart contract.

3. Reentrancy Vulnerability Modification

To demonstrate a reentrancy attack, a vulnerable contract `ReentrancyVulnerableLoan.sol` was introduced. In the vulnerable implementation, the `withdraw()` function transfers Ether to the caller *before* updating the user's balance:

```
(bool sent, ) = payable(msg.sender).call{value: amount}("");
require(sent, "TransferFailed");
balances[msg.sender] = 0; // State is updated AFTER the external call
```

An attacker contract (`ReentrancyAttacker.sol`) exploits this by implementing a `receive()` fallback function that recursively calls `withdraw()`. Because the balance is not set to 0 until the very end, the attacker can repeatedly withdraw funds until the contract is drained.
Resolution in Main Contracts: The main protocol (e.g., `FundingPool.sol`) avoids this entirely by strictly adhering to the **Checks-Effects-Interactions** pattern. Internal state balances (like `totalContributed` and `pendingInterestGain`) are deducted *before* any external `.call{value: amount}("")` is executed.

4. Discussion on Malicious Strategies

Even without technical vulnerabilities like reentrancy, the protocol's economic rules present potential attack vectors for malicious actors:
Majority Voting Takeover (51% Attack)
Since voting power is proportional to a contributor's unlocked balance, a "whale" contributor (or a coordinated syndicate) holding more than 50% of the pool's liquidity can unilaterally approve or reject any loan. A malicious whale could execute the following strategy:

1. The whale deposits a massive amount of ETH into the funding pool.
2. The whale uses a secondary wallet (Sybil account) to propose a loan for a large portion of the pool's liquidity, offering a high interest rate.

3. The borrower account points to a rich Bitcoin address (since ownership of the BTC is not verified by a cryptographic signature, any publicly rich BTC address can be used to pass the liquidity check).
4. The whale uses their majority voting power to force the loan's approval.
5. Once funded, the borrower (the whale's alt account) absconds with the ETH and deliberately defaults on the loan.
6. The honest minority contributors lose their locked funds, while the whale effectively steals the pool's liquidity.

This highlights a fundamental flaw in using unverified, non-locked Bitcoin balances as collateral combined with plutocratic voting systems.

5. Brief User Manual

Follow these steps to set up, deploy, and simulate the Decentralised Lending Service:

Prerequisites

- Place the Bitcoin blockchain files (`blk00000.dat` and `blk00001.dat`) in a `bitcoin/` directory at the root of the project to allow the Oracle to parse real UTXO data.
- **Node.js** & **npm**
- **Python 3.10+** (dependencies in `python/requirements.txt`)
- A local **Geth** node running the provided genesis block.

1. Installation & Environment Setup

Run the setup script to install Node.js dependencies, configure the Python virtual environment, compile the contracts, and automatically download the required Geth binary.

```
./setup.sh
```

2. Start the Private Blockchain

In a new terminal window, initialize and start the local Geth node (ensure the pre-funded account is unlocked to allow block mining):

```
cd private-chain
./geth --datadir=./data init project2526genesis.json
./geth --datadir=./data --keystore=./keystore --networkid=202526 --http --http.addr=127.0.0
  --http.port=8545 --http.api=eth,net,web3 --mine \
  --miner.etherbase=d278d247a52c550508ea2b2c9321d816238fb523 \
  --unlock d278d247a52c550508ea2b2c9321d816238fb523 \
  --password password.txt --allow-insecure-unlock --ipcdisable
```

3. Deploy the Smart Contracts

In a new terminal, deploy the protocol to the private chain. This will generate necessary `.env` configuration files for the Python services.

```
./.venv/bin/python python/setup.py
```

4. Run the Off-Chain Services

The protocol relies on two Python services to function automatically. Run these in separate terminals:

- **Bitcoin Oracle**: Listens for on-chain update requests and fetches balances from the `.dat` files.

  ```
  ./.venv/bin/python python/oracle/oracle_service.py
  ```

- **Auto-Voter Bot**: Automatically votes "YES" on all new loan proposals to ensure loans pass seamlessly.

  ```
  ./.venv/bin/python python/voter/auto_voter.py
  ```

5. Simulate the Workflow

In a final terminal, run the simulation script which performs a full lifecycle (Deposit → Loan Proposal → Oracle Update → Voting → Execution → Repayment).

```
./.venv/bin/python python/simulate.py
```

6. Run the Test Suite

To run the full suite of Hardhat tests (including gas profiling, trace compliance, and security tests):

```
npm test
```